# Deep Learning for Economics and Finance

## Random search and Hyperband for DEQN architectures

Simon Scheidegger

HEC, University of Lausanne

Materials: https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

# Roadmap of This Lecture

**I. Motivation**

- Why architecture matters
- The hyperparameter space
- Limits of manual tuning

**II. Search Methods**

- Grid vs. random search
- Bayesian optimization
- Hyperband / successive halving
- Method comparison

**III. Implementing the Search**

- Defining a search space
- Random Search + SHA from scratch
- Tool landscape (footnote)

**IV. Example & Wrap-up**

- Genz-function approximation
- Best practices
- Connection to DEQNs

Hands-on: `02_NAS_Random_Search_10D.ipynb` (10-D random search)

`03_NAS_RandomSearch_Hyperband.ipynb` (Random Search + SHA)

# Learning objectives

**By the end of this lecture you should be able to:**

- Diagnose architecture sensitivity and quantify how network depth and width affect approximation error
- Compare grid search, random search, Bayesian optimization, and Hyperband on their computational cost and sample efficiency
- Implement random search and successive halving from scratch in plain Python
- Apply NAS to function approximation problems and choose appropriate search spaces for economic models
- Select production tools (KerasTuner, Optuna, Ray Tune) and apply best practices to scale architecture search efficiently

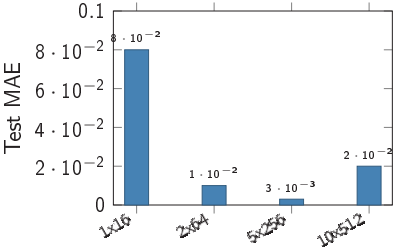
# Why Architecture Matters

**Same data, different architectures:**

- 1 layer, 16 units $\Rightarrow$ MAE $= 0.08$
- 2 layers, 64 units $\Rightarrow$ MAE $= 0.01$
- 5 layers, 256 units $\Rightarrow$ MAE $= 0.003$
- 10 layers, 512 units $\Rightarrow$ MAE $= 0.02$ (overfit)

Architecture choice can matter more than training duration.

*Illustrative numbers from a depth/width sweep on the companion regression task; the from-scratch search in Notebook 03 reproduces the same ordering on a different testbed.*



# The Hyperparameter Space

**Architectural hyperparameters:**

- Number of hidden layers $L \in \{1, \ldots, 5\}$
- Units per layer $n_\ell \in \{32, 64, \ldots, 256\}$ (8 values)
- Activation: ReLU, tanh, swish

**Training hyperparameters:**

- Learning rate $\eta$ log-uniform on $[10^{-4}, 10^{-2}]$ (20-point grid)
- Batch size $B \in \{16, 32, 64, 128\}$
- Epochs, regularization, ...

**Combinatorial explosion**

Even this modest search space:

$$|\mathcal{H}| = 5 \times 8 \times 3 \times 20 \times 4 = 9{,}600$$

- Each trial: minutes to hours
- Full grid: infeasible

# Manual Tuning

**The "grad-student descent" approach:**

1. Pick an architecture (intuition / paper)
2. Train, evaluate, adjust one thing
3. Repeat until deadline

**Problems:**

- Doesn't scale beyond a few hyperparameters
- Human bias: we try what we already know
- Not reproducible: results depend on the researcher
- Misses interactions between hyperparameters

**Key insight**

We need **automated, systematic** methods to search the hyperparameter space efficiently.

# Automated Search Strategies

# Grid Search

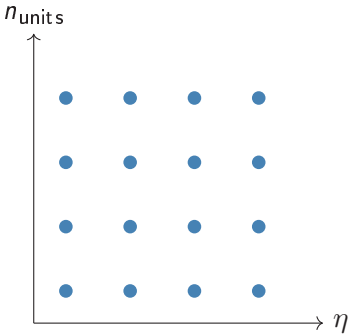
**Idea:** Enumerate all combinations on a regular grid.

## Algorithm

1. Define discrete set for each HP
2. Evaluate *every* combination
3. Return the best

**Cost:** $\mathcal{O}(k^d)$ — exponential in dimension $d$.

**Pros:** Simple, embarrassingly parallel.
**Cons:** Wastes budget on unimportant dimensions.




16 evaluations

# Random Search

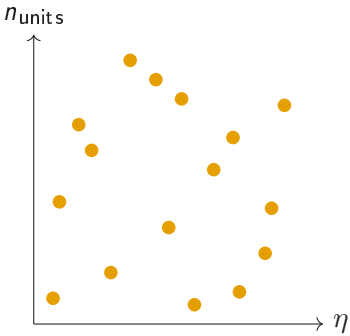
**Idea:** Sample configurations uniformly at random.

**Key result (Bergstra & Bengio, 2012)**

When only a *few* dimensions matter, random search finds good configurations with fewer trials than grid search.

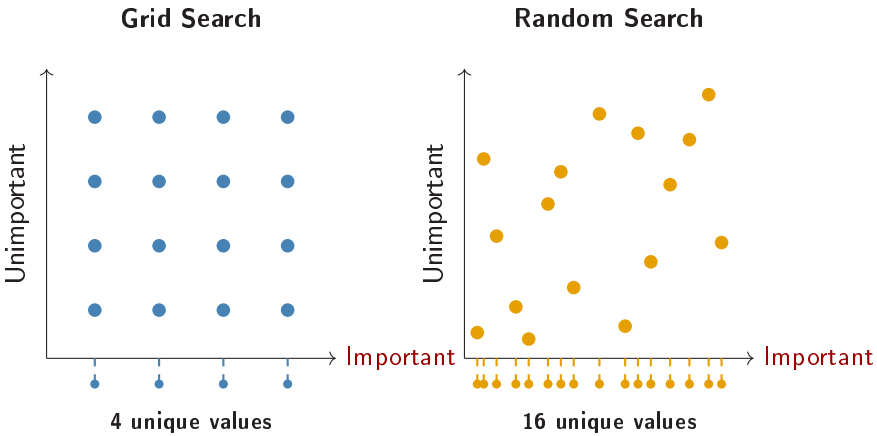
**Why?** Random points explore each dimension *independently* — more unique values per HP.

**Cost:** $\mathcal{O}(n)$ — linear in budget $n$.




16 evaluations

# Grid vs. Random — Why Random Wins




When one dimension matters, the grid wastes most evaluations on the unimportant axis; random search samples the important axis more densely.

# Bayesian Optimization

**Idea:** Build a *surrogate model* of $f(\text{HP}) \to \text{loss}$ and use it to choose the next trial *intelligently*.

1. Fit a Gaussian process (GP) to observed trials
2. Compute an acquisition function (e.g., Expected Improvement)
3. Evaluate the HP with highest acquisition value
4. Update the GP and repeat

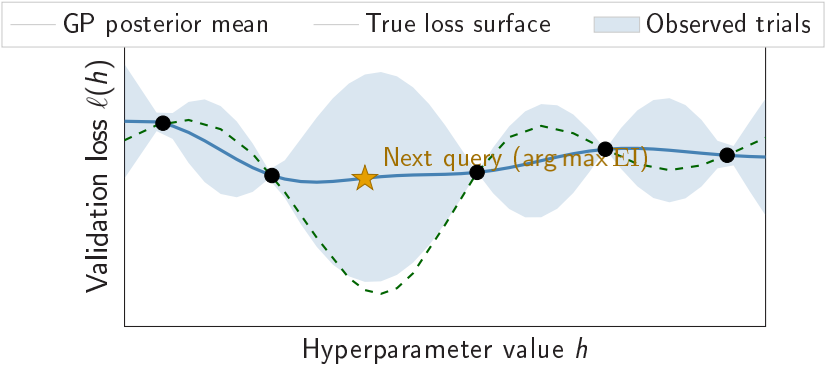
**Key advantage:** Sample-efficient — exploits structure in the response surface.

## Expected Improvement

$$\text{EI}(x) = \mathbb{E}[\max(f^* - f(x), 0)]$$

- $f^*$: best observed value
- Balances *exploration* (high uncertainty) and *exploitation* (low predicted loss)

# Bayesian Optimization — Visual




The GP posterior mean *interpolates* the five observations and is *wide* in unexplored gaps. The Expected-Improvement acquisition function peaks at $h \approx 3.75$, trading off the high posterior variance there against the moderately low predicted mean.

# Hyperband

**Idea**: Start many configurations cheaply; stop the worst early and give more resources to promising ones.

## Successive Halving (SHA, $\eta = 3$)

1. Start $n$ random configs with budget $b$
2. Train each for $b$ epochs, evaluate
3. Keep the top $1/\eta$ of configs (discard the rest)
4. Multiply the budget by $\eta$, repeat

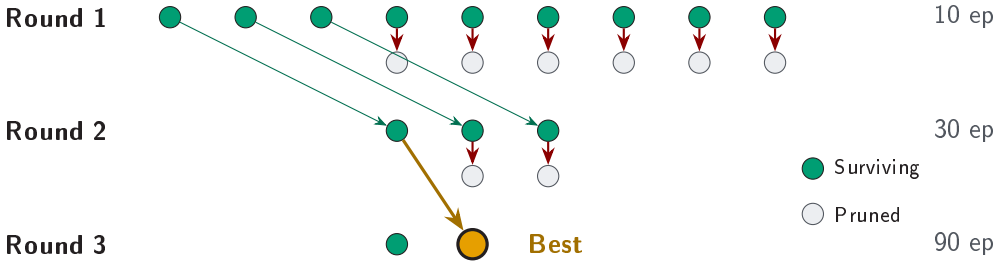
**Hyperband** = SHA with several brackets (different $n/b$ trade-offs), each bracket itself being a stage cascade like $9 \to 3 \to 1$ above. The usual default is $\eta = 3$.
**Resource use**: many cheap trials; full budgets only for survivors.

**Key insight**:

- Bad configs identifiable *early*
- No need for 200 epochs to know $\eta = 10$ diverges
- Exploits correlation between early and late performance

Li et al. (2018): large speedups when early and late performance are correlated.

# Hyperband — Successive Halving




$9 \to 3 \to 1$ (factor $\eta = 3$): total cost $= 9 \times 10 + 3 \times 30 + 1 \times 90 = 270$ epoch-equivalents (vs. $9 \times 90 = 810$ for full training).

**Bracket vs. schedule.** This $9 \to 3 \to 1$ cascade is one SHA bracket. Hyperband itself runs several brackets in parallel, each with a different starting $(n, b)$ trade-off, hedging across the unknown correlation between early- and late-training performance.

# Method Comparison

| Property | Grid | Random | Bayesian | Hyperband |
|---|---|---|---|---|
| Computational cost | $\mathcal{O}(k^d)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n \log n)$ |
| Sample efficiency | Low | Medium | High | High |
| Parallelisable | Fully | Fully | Limited | Partially |
| Handles interactions | No | Partially | Yes | No |
| Early stopping | No | No | No | Yes |
| Implementation | Trivial | Trivial | Moderate | Moderate |

**Practical advice:**

- **Quick exploration:** Random search (strong baseline)
- **Limited budget:** Bayesian optimization (sample-efficient)
- **Large search space:** Hyperband (aggressive early stopping)

# Implementing the Search

*Random Search and Successive Halving, from scratch*

# Defining a Search Space

## A plain dict — no library required

```
SPACE = {
    "layers":     [1, 2, 3, 4, 5],
    "units":      [32, 64, 96, 128, 160, 192, 224, 256],
    "activation": ["relu", "tanh", "swish"],
    "lr":         ("log_uniform", 1e-4, 1e-2),
}
def sample_config(rng):
    return {
        "layers":     int(rng.choice(SPACE["layers"])),
        "units":      int(rng.choice(SPACE["units"])),
        "activation": str(rng.choice(SPACE["activation"])),
        "lr":         float(10 ** rng.uniform(-4, -2)),
    }
```

No `hp.Int` / `hp.Float` abstraction — the search space is a dict, the sampler is one function.

# Random Search from Scratch

## 30 trials, single function

```
def random_search(n_trials=30, epochs=50, seed=0):
    rng = np.random.default_rng(seed)
    records = []
    for t in range(n_trials):
        cfg = sample_config(rng)
        model = build_model(cfg)
        h = model.fit(X_tr, y_tr, epochs=epochs,
                      validation_data=(X_val, y_val), verbose=0)
        records.append((cfg, min(h.history["val_loss"])))
    return min(records, key=lambda r: r[1])
```

Independent uniform draws; $\mathcal{O}(N)$ cost, embarrassingly parallel (Bergstra & Bengio, 2012).

# Successive Halving from Scratch

## $n_0 = 27$, $\eta = 3$, three rounds

```
def successive_halving(n0=27, eta=3, r0=8, seed=0):
    rng = np.random.default_rng(seed)
    survivors = [(sample_config(rng), None) for _ in range(n0)]
    r = r0
    while len(survivors) > 1:
        scored = [(cfg, train_for(cfg, epochs=r)) for cfg, _ in survivors]
        scored.sort(key=lambda s: s[1])           # ascending val loss
        keep   = max(1, len(scored) // eta)
        survivors = scored[:keep]
        r *= eta                                  # 8 -> 24 -> 72 epochs
    return survivors[0]
```

Notebook schedule: 27 candidates @ 8 ep $\rightarrow$ 9 @ 24 ep $\rightarrow$ 3 @ 72 ep $\rightarrow$ 1 winner. This is 648 config-epochs, versus 1500 for 30 random-search trials at 50 epochs each: about 2.3$\times$ less compute while testing a comparable number of architectures.

# Production Tooling (footnote)

We just wrote both algorithms in ∼25 lines of Python. Real projects rarely hand-roll the loop; established libraries wrap (and parallelise) the same algorithms behind a uniform API:

| Library | Algorithms | Notes |
|---|---|---|
| KerasTuner | Random, Bayesian, Hyperband | Tight Keras integration |
| Optuna | TPE, CMA-ES, Hyperband, NSGA-II | Framework-agnostic; popular default |
| Ray Tune | All of the above + ASHA, PBT | Distributed by design |
| Hyperopt | TPE, Random | The original TPE reference |
| Ax / BoTorch | Bayesian (multi-objective, MOO) | Meta; PyTorch-native GPs |
| NNI | Random, Bayesian, evolutionary, NAS | Microsoft; full graph-NAS support |

**Why teach algorithms, not wrappers.** APIs change; the search loops (Random, SHA/Hyperband, GP+EI, TPE) persist.

# Example: Genz Function

## Approximation

# Experiment Setup

**Target function:** Genz Gaussian on $[0,1]^2$

$$f(\mathrm{x}) = \exp\Big(-\sum_{i=1}^{d} c_i^2 (x_i - w_i)^2\Big)$$

**Data:**

- $d = 2$, uniform random samples
- $n_{\text{train}} = 1{,}000$, $n_{\text{test}} = 2{,}000$
- Loss: MSE, metric: test MAE

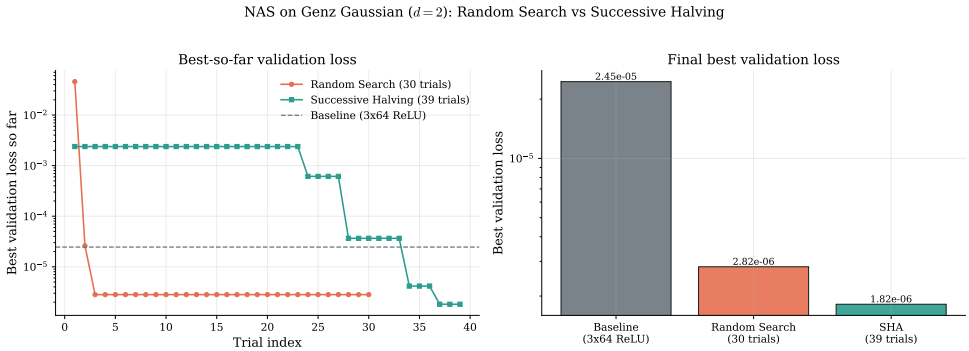
**Search space:**

- Layers: 1–5
- Units/layer: 32–256 (step 32)
- Activation: ReLU, tanh, swish
- LR: $10^{-4}$–$10^{-2}$ (log scale)

**Baseline:** $3 \times 64$ ReLU MLP, $\eta = 10^{-3}$

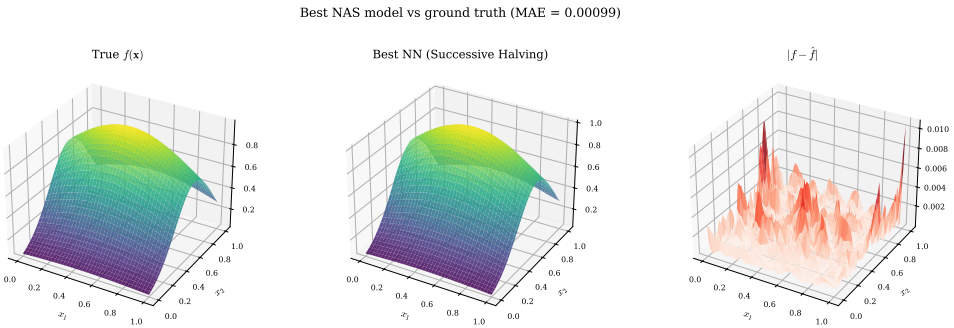
**Budget:** 30 trials per method

# Search Results (Random Search vs. SHA on a 2-D Genz Gaussian)




Generated by `03_NAS_RandomSearch_Hyperband.ipynb`. Random Search uses 30 full trials; SHA uses 27 initial candidates with $\eta = 3$. Both reach MAE of order $10^{-3}$ from a baseline near $4 \cdot 10^{-3}$; SHA uses about $2.3\times$ less config-epoch budget in this seeded run.

# Best Model vs. Ground Truth




Generated by `03_NAS_RandomSearch_Hyperband.ipynb`. Left to right: the true Genz target, the best NAS-found NN approximation, and the absolute error map on $[0, 1]^2$. No placeholder shapes; this is the actual notebook output. The companion `02_NAS_Random_Search_10D.ipynb` offers a 10-D testbed (search loop in plain Python, model in TF/Keras).

# Best Practices for NAS

**Search space design:**

- Start small, expand if needed
- Use log-scale for learning rate
- Constrain to reasonable ranges
- Include the current best as a point

**Budget allocation:**

- Diminishing returns after $\sim$30 trials
- Hyperband: set `factor=3` (default)
- Use early stopping callbacks

**Common pitfalls:**

- Search space too large $\Rightarrow$ wasted budget
- Overfitting to validation set $\Rightarrow$ use holdout test
- Ignoring training stability $\Rightarrow$ check learning curves

**Rule of thumb**

If manual tuning took $N$ trials, automated search with $3N$ trials will likely find something better.

# Connection to Economics

**Where NAS matters in computational economics:**

1. **Surrogate models:** Replace expensive simulations (e.g., climate IAMs) with NNs — architecture determines approximation quality.
2. **DSGE policy functions:** Deep equilibrium nets (Lecture 03) use NNs to approximate policy/value functions. Architecture search over the policy network can improve convergence.
3. **High-dimensional problems:** As $d$ grows (many state variables), the architecture must scale. NAS automates this adaptation.
4. **Reproducibility:** Automated search documents *which* architectures were tried, making results more transparent than "we chose 3 layers with 64 units."

# Summary

**Key takeaways:**

1. Architecture choice strongly affects NN performance
2. Random search is a strong baseline — always try it first
3. Bayesian optimization is most sample-efficient
4. Hyperband / SHA offers the best speed/quality trade-off
5. Production: KerasTuner, Optuna, Ray Tune, Ax all wrap these

**References:**


- Elsken, Metzen & Hutter (2019) — *NAS: A Survey*, JMLR 20(55); see `readings/links_by_lecture/lecture_05.md`.
- Bergstra & Bengio (2012). *Random search.* JMLR 13.
- Li et al. (2018). *Hyperband.* JMLR 18(185).
- Garnett (2023). *Bayesian optimization.* CUP.
- Snoek et al. (2012). *Practical Bayesian optimization.* NeurIPS.